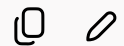




PDF

Lab_4_VibeCoding_Student_handout

I'm working on a Pygame lab: a marble tilt maze. The marble accelerates toward the mouse cursor, bounces off walls, and there is a countdown timer. Extract the files from the git link:



As part of your assignment detailed in Lab_4_VibeCoding_Student_handout.pdf, you are required to use AI tools to fix the broken code. PDF

Here is the updated `_resolve_wall_collisions` method using true circle-vs-rectangle collision logic:

Python



```
import math

def _resolve_wall_collisions(self):
    bounce_factor = 0.3

    for wall in self.walls:
        # Find the closest point on the wall rectangle to the marble's center
        closest_x = max(wall.left, min(self.x, wall.right))
        closest_y = max(wall.top, min(self.y, wall.bottom))

        # Calculate the distance from the marble's center to this closest point
        dx = self.x - closest_x
        dy = self.y - closest_y
        distance = math.hypot(dx, dy)

        # Check if there is a collision
        if distance < self.radius:
            if distance == 0:
                # Center of the marble is inside the rectangle; find the closest
                dist_left = self.x - wall.left
```

```

dist_right = wall.right - self.x
dist_top = self.y - wall.top
dist_bottom = wall.bottom - self.y

min_dist = min(dist_left, dist_right, dist_top, dist_bottom)

if min_dist == dist_left:
    nx, ny = -1.0, 0.0
    overlap = self.radius + dist_left
elif min_dist == dist_right:
    nx, ny = 1.0, 0.0
    overlap = self.radius + dist_right
elif min_dist == dist_top:
    nx, ny = 0.0, -1.0
    overlap = self.radius + dist_top
else:
    nx, ny = 0.0, 1.0
    overlap = self.radius + dist_bottom
else:
    # Normal vector pointing from the wall to the marble
    nx = dx / distance
    ny = dy / distance
    overlap = self.radius - distance

# Push the marble out along the contact normal
self.x += nx * overlap
self.y += ny * overlap

# Reflect the velocity component going into the wall
dot_product = self.vx * nx + self.vy * ny
if dot_product < 0:
    self.vx -= (1 + bounce_factor) * dot_product * nx
    self.vy -= (1 + bounce_factor) * dot_product * ny

```

Why the old code failed:

Pygame's standard `colliderect` function treats all objects as Axis-Aligned Bounding Boxes (AABBs)—meaning it treated your marble like a square. Because a bounding square has 90-degree corners that extend past the visual curve of the marble, those invisible corners intersected the wall before the circle's edge did. This resulted in the marble bouncing off empty space. True radial collision mathematically checks the exact distance from the center to the closest edge, ignoring the square's bounding box completely.

Task 2: Add a game over screen. Instead of printing to the console, draw a semi-transparent overlay showing "Maze Solved!" with the finish time, or



game_engine.py

Python



```
# 1. In your __init__ method, initialize the game over states:
self.game_over = False
self.victory = False
self.quit_requested = False
self.final_time = 0.0

# 2. In your update() method, freeze logic on game over and set end states:
def update(self, dt):
    if self.game_over:
        return # Freeze timer and marble logic

    # [Keep existing movement and timer logic here]

    # Replace previous console prints with state flags when target is reached:
    if self._reached_target(): # Adjust to your specific victory check
        self.game_over = True
        self.victory = True
        self.final_time = self.time_left # Adjust to your specific timer variable

    # Trigger game over on timeout:
    if self.time_left <= 0:
        self.game_over = True
        self.victory = False

# 3. In your event handling method, catch exit keys during the game over screen
def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_ESCAPE, pygame.K_Q):
            self.quit_requested = True
        return

    # [Keep existing event handling here]

# 4. At the end of your draw() method, render the semi-transparent overlay:
```

```
def draw(self, screen):
    # [Keep existing rendering here]

    if self.game_over:
        overlay = pygame.Surface(screen.get_size(), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))

        font_large = pygame.font.SysFont(None, 64)
        font_small = pygame.font.SysFont(None, 36)

        if self.victory:
            msg = font_large.render(f"Maze Solved! Time: {self.final_time:.2f}s", True, (255, 255, 255))
        else:
            msg = font_large.render("Time's Up!", True, (255, 100, 100))

        exit_msg = font_small.render("Press Q or ESC to exit", True, (200, 200, 200))

        screen.blit(msg, msg.get_rect(center=(screen.get_width() // 2, screen.get_height() // 2)))
        screen.blit(exit_msg, exit_msg.get_rect(center=(screen.get_width() // 2, screen.get_height() // 2)))
```

main.py

Python



```
# Inside your main application loop, check the flag to exit cleanly:
while running:
    # [Keep existing event pumping, engine.update(), and engine.draw() here]

    if engine.quit_requested:
        running = False
        break

    # [Keep existing display.flip() and clock.tick() here]
```

Task 3: After the end screen, let the player replay by choosing a difficulty:

game_engine.py

Python



```
# 1. Add the new restart method to handle state and difficulty resets:
def restart(self, difficulty="Medium"):
    self.difficulty = difficulty
    self.game_over = False
    self.victory = False
    self.quit_requested = False

    # Adjust these attribute names and values to match your specific physics values
    if difficulty == "Easy":
        self.tilt_strength = 500.0 # Lower acceleration
        self.friction = 0.90 # Higher friction (slows down faster)
        self.time_left = 60.0 # More time
    elif difficulty == "Hard":
        self.tilt_strength = 1500.0 # Higher acceleration
        self.friction = 0.999 # Lower friction (slides more)
        self.time_left = 15.0 # Less time
    else: # Medium (Current defaults)
        self.tilt_strength = 1000.0
        self.friction = 0.98
        self.time_left = 30.0

    # Reset marble position and velocity
    self.x, self.y = self.start_x, self.start_y
    self.vx, self.vy = 0.0, 0.0

# 2. In your __init__ method, save the spawn point and initialize via restart:
# (Replace your existing velocity, time, and game_over declarations with this)
self.start_x = self.x
self.start_y = self.y
self.restart("Medium")

# 3. Update handle_event to catch difficulty selections during game over:
def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_ESCAPE, pygame.K_q):
                self.quit_requested = True
```

```

        elif event.key == pygame.K_1:
            self.restart("Easy")
        elif event.key == pygame.K_2:
            self.restart("Medium")
        elif event.key == pygame.K_3:
            self.restart("Hard")
    return

# [Keep existing gameplay event handling here]

# 4. In your draw() method, add the difficulty to the HUD and update the end text
def draw(self, screen):
    # [Keep existing background and maze drawing here]

    # --- HUD Addition ---
    font_hud = pygame.font.SysFont(None, 28)
    diff_text = font_hud.render(f"Difficulty: {self.difficulty}", True, (255, 255, 255))
    screen.blit(diff_text, (10, 40)) # Adjust Y coordinate to not overlap your

    if self.game_over:
        # [Keep overlay rendering and victory/failure msg from Task 2]

        # Replace the old exit_msg with the new options:
        exit_msg = font_small.render("Press 1: Easy | 2: Medium | 3: Hard | Q/E", True, (255, 255, 255))
        screen.blit(exit_msg, (10, 100))

        # [Keep existing blit positioning for msg and exit_msg here]

```

Task 4: Add sound effects for bouncing off a wall, reaching the goal, and the timer running out. Don't add any new dependencies or audio files: synthesize the



game/sound.py

Python



```

import pygame
import array
import math
import random

class SoundManager:
    def __init__(self):
        self.enabled = False
        try:
            # Initialize mixer if not already done by main.py
            if not pygame.mixer.get_init():
                pygame.mixer.init()

            init_info = pygame.mixer.get_init()
            if init_info:
                self.sample_rate = init_info[0]
                self.format = init_info[1]
                self.channels = init_info[2]

                # Support the standard 16-bit signed format
                if abs(self.format) == 16:
                    self.enabled = True
        except Exception:
            pass

        self.bounce_sound = None
        self.goal_sound = None
        self.timeout_sound = None
        self.last_bounce_time = 0

        if self.enabled:
            self.bounce_sound = self._generate_tone(frequency=150, duration=0.6)
            self.goal_sound = self._generate_tone(frequency=880, duration=0.4, volume=0.5)
            self.timeout_sound = self._generate_tone(frequency=150, duration=0.6, volume=0.5)

    def _generate_tone(self, frequency, duration, volume=1.0, wave_type='sine'):
        num_samples = int(self.sample_rate * duration)
        buf = array.array('h') # 'h' for signed 16-bit integers
        max_amp = int(32767 * volume)

        for i in range(num_samples):
            t = i / self.sample_rate

            if wave_type == 'sine':
                sample = int(max_amp * math.sin(2 * math.pi * frequency * t))
            elif wave_type == 'sawtooth':
                # Descending pitch effect for time out
                freq = frequency - (100 * t)
                sample = int(max_amp * (2 * (t * freq - math.floor(0.5 + t * frequency))))

```

```
elif wave_type == 'noise':
    sample = random.randint(-max_amp, max_amp)

    # Quick 10ms envelope to avoid audio clipping/popping at the ends
    envelope = 1.0
    fade_samples = int(self.sample_rate * 0.01)
    if i < fade_samples:
        envelope = i / fade_samples
    elif i > num_samples - fade_samples:
        envelope = (num_samples - i) / fade_samples

    sample = int(sample * envelope)

    # Append sample for each channel (usually stereo)
    for _ in range(self.channels):
        buf.append(sample)

return pygame.mixer.Sound(buffer=buf)

def play_bounce(self, impact_speed):
    if not self.enabled or not self.bounce_sound:
        return

    current_time = pygame.time.get_ticks()
    # Short cooldown to prevent buzzing from continuous corner collisions
    if current_time - self.last_bounce_time < 50:
        return

    # Ignore tiny resting contacts
    if impact_speed < 15:
        return

    # Scale volume (assume 500 is a hard hit)
    vol = min(1.0, max(0.1, impact_speed / 500.0))
    self.bounce_sound.set_volume(vol)
    self.bounce_sound.play()
    self.last_bounce_time = current_time

def play_goal(self):
    if self.enabled and self.goal_sound:
        self.goal_sound.play()

def play_timeout(self):
    if self.enabled and self.timeout_sound:
        self.timeout_sound.play()
```

Python



```
# 1. Add the import at the top of the file:
from game.sound import SoundManager

# 2. In your __init__ method, initialize the manager:
def __init__(self, ...):
    # [Keep existing variable initializations]
    self.sound_manager = SoundManager()

# 3. In update(), play sounds right before setting the game_over flag:
def update(self, dt):
    if self.game_over:
        return

    # [Keep existing movement and timer logic here]

    if self._reached_target():
        self.sound_manager.play_goal()
        self.game_over = True
        self.victory = True
        self.final_time = self.time_left

    if self.time_left <= 0:
        self.sound_manager.play_timeout()
        self.game_over = True
        self.victory = False

# 4. In _resolve_wall_collisions (from Task 1), add the bounce sound call:
# Inside your check if the dot_product is negative (moving into the wall)
    # Reflect the velocity component going into the wall
    dot_product = self.vx * nx + self.vy * ny
    if dot_product < 0:
        # The dot product is negative; flipping it gives the impact speed
        impact_speed = -dot_product
        self.sound_manager.play_bounce(impact_speed)

        self.vx -= (1 + bounce_factor) * dot_product * nx
        self.vy -= (1 + bounce_factor) * dot_product * ny
```